

HW3

Masum Billah

2026-04-03

Q1

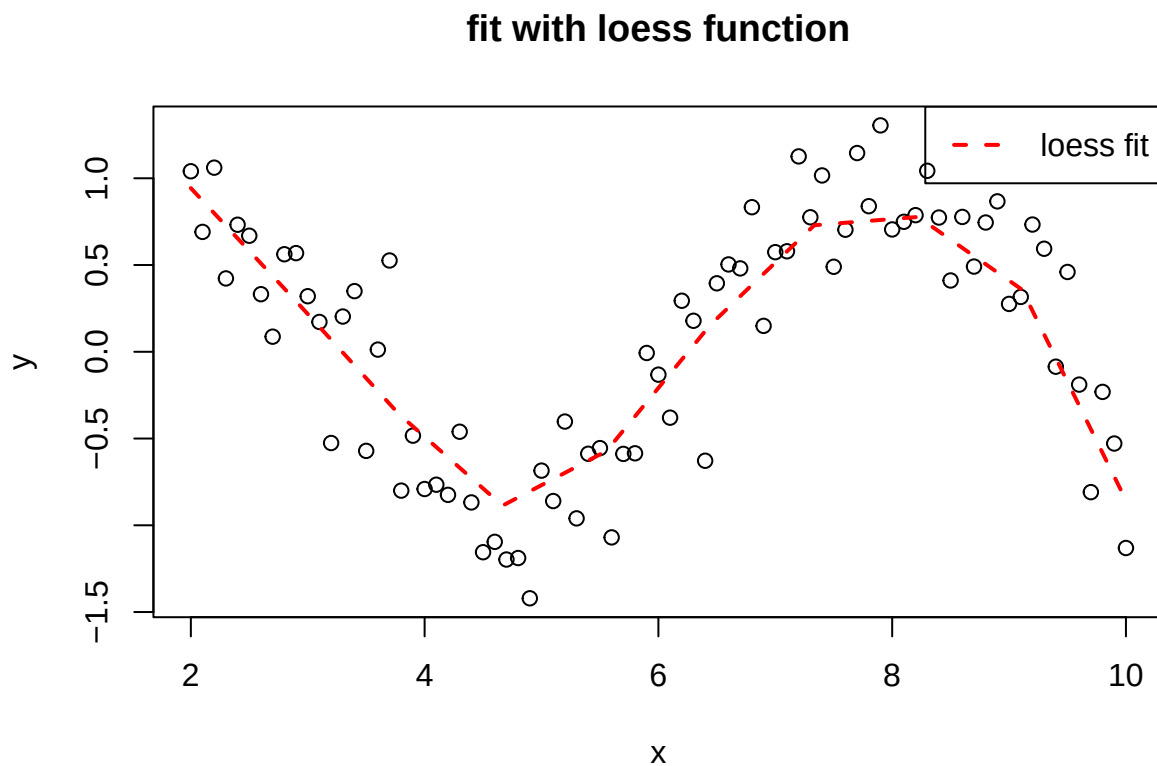
```
myloess <- function(dat, lamda, numpts){  
  
  # The function takes data, a bandwidth parameter, and the number of x-points as input,  
  # and returns a matrix containing the numpts x-values and the corresponding evaluated y-values at each point.  
  
  # dat      : x and y values  
  # lamda    : bandwidth parameter  
  # numpts   : number of x-points  
  
  dat = dat[order(dat[, 1]), ] # sort data based on x-values  
  
  x = dat[, 1]  
  y = dat[, 2]  
  
  p = seq(from=min(x), to=max(x), length.out=numpts) # generate equally spaced points over the range of x  
  s = 2 * (lamda * sd(x))^2 # calculate the exponential (numerator) part of the normal distribution.  
  y_loess = data.frame() # vector to store evaluated y-values at each point.  
  
  for (x0 in p) {  
    dist = (x - x0)**2 # calculate distance  
    w = exp(-(dist/s)) # convert to weight  
    model = lm(y ~ x, weights = w) # fit weighted least square  
    #print(summary(model))  
    y0 = predict(model, newdata = data.frame(x = x0)) # evaluate at x0  
    y_loess = rbind(y_loess, data.frame(x0, y0))  
  }  
  
  {  
    plot(x, y, xlab='x', ylab='y') # plot the original data points  
    lines(y_loess$x0, y_loess$y0, col='red', lty=2, lwd=2) # plot the fitted model  
    title(main="fit with loess function")  
    legend("topright", legend = "loess fit", col = "red", lty = 2, lwd = 2)  
  }  
  
  return(y_loess)  
}
```

Case 1

Here, I simulate $\sin(x)$ as the true function and add Gaussian noise $N(\mu=0, \text{sd}=0.3)$ to generate the response y . The `myloess` function is then applied, using 10 evaluation points and $\lambda = 0.2$, to fit the data via weighted least squares.

```
xpts = seq(2, 10, 0.1) # x-values
f = sin(xpts) # true function
ypts = f + rnorm(length(xpts), 0, 0.3) # y-values
dat = data.frame(xpts, ypts)

y_loess = myloess(dat, 0.2, 10)
```

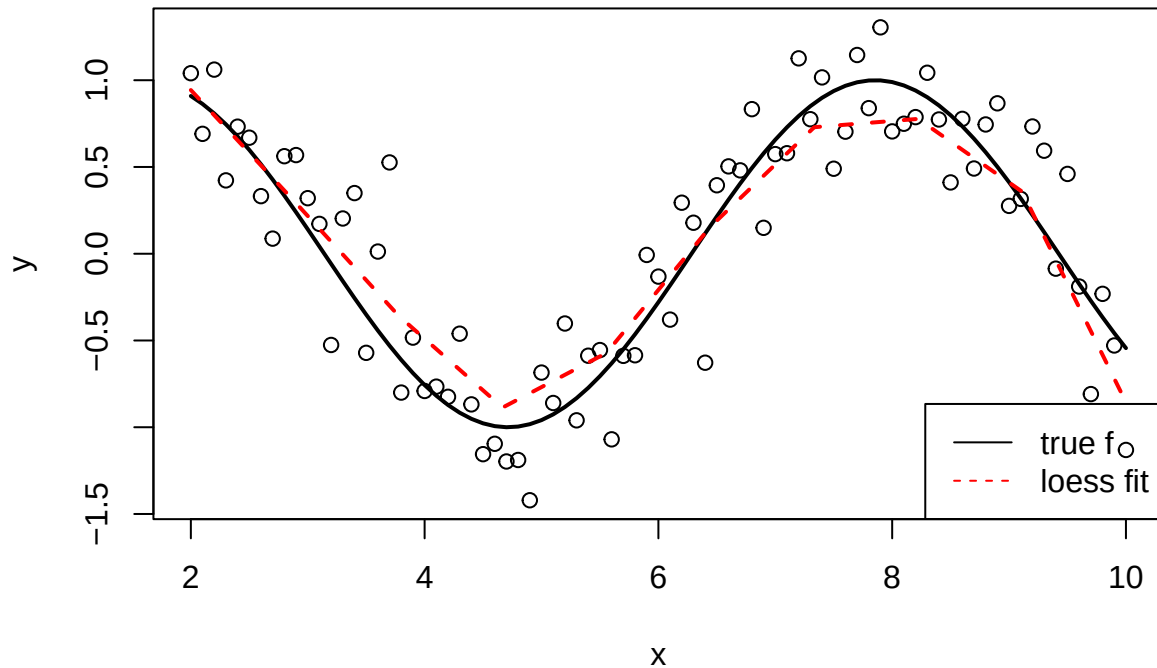


The above figure shows the original data points and the fitted LOESS curve. Here, we can see that, the LOESS curve closely follows the data pattern.

```
# plot
{
plot(xpts, ypts, xlab='x', ylab='y') # plot the original data points
lines(xpts, f, col="black", lty=1, lwd=2) # plot the true function
lines(y_loess$x0, y_loess$y0, col='red', lty=2, lwd=2) # plot the fitted model

title(main="fit with loess function")
legend("bottomright", legend = c("true f", "loess fit"), col = c("black", "red"), lty = c(1, 2))
}
```

fit with loess function



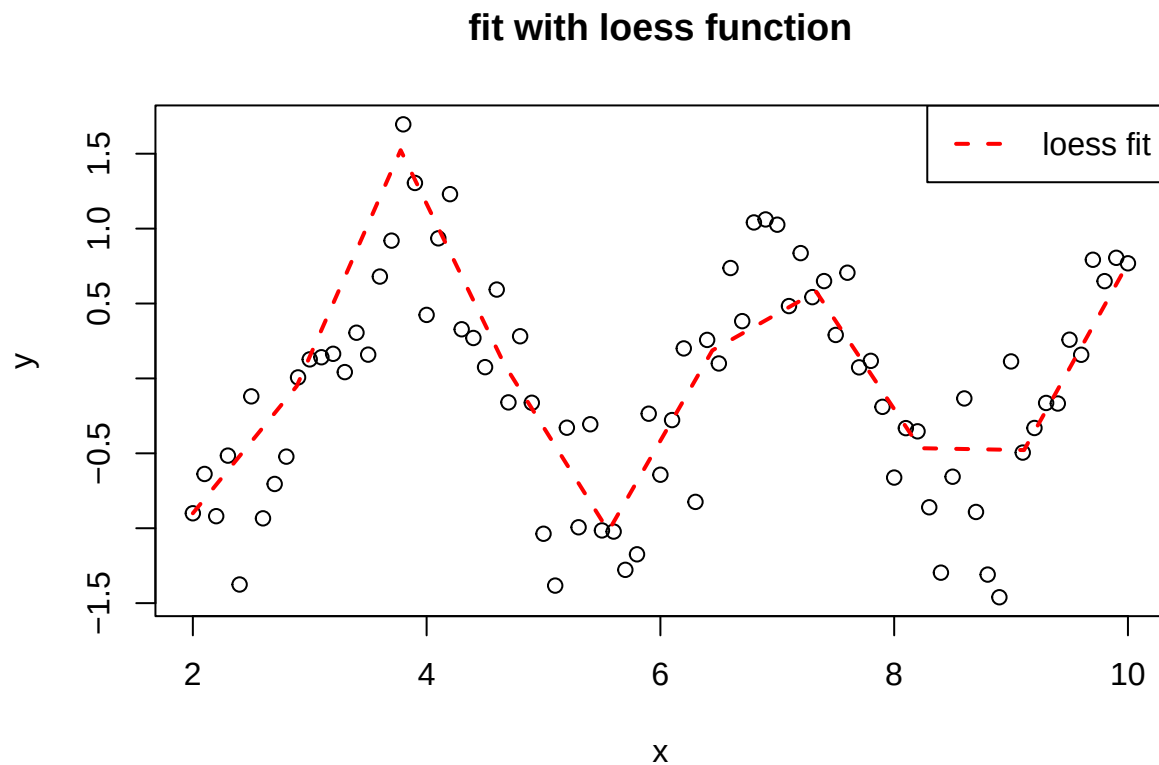
The above figure shows the original data points, the true regression function, and the fitted LOESS curve. The LOESS curve closely follows the true regression function, effectively capturing the overall structure of the data.

Case 2

For case 2, I simulate $\sin(2x)$ as the true function and add Gaussian noise $N(\mu=0, \text{sd}=0.3)$ to generate the response y . The `myloess` function is then applied, using 10 evaluation points with $\text{lamda} = 0.01$, to fit the data via weighted least squares.

```
xpts = seq(2, 10, 0.1) # x-values
f = sin(2*xpts) # true function
ypts = f + rnorm(length(xpts), 0, 0.3) # y-values
dat = data.frame(xpts, ypts)

y_loess = myloess(dat, 0.01, 10)
```

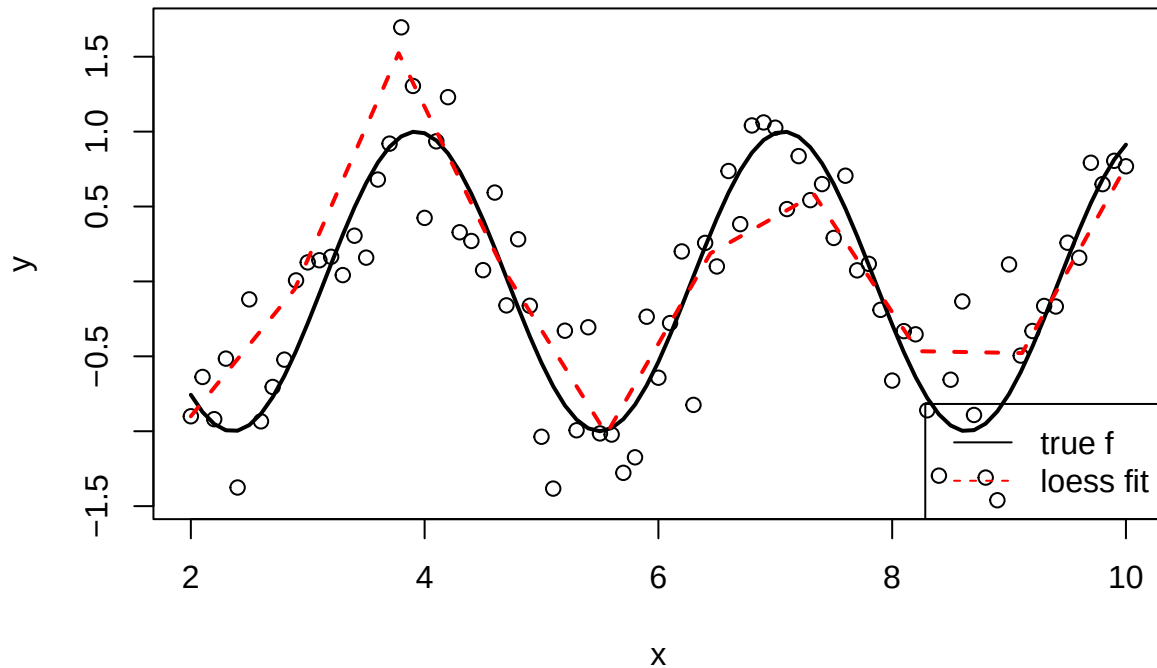


The figure shows the original data points and the fitted LOESS curve. Using 10 evaluation points with $\lambda = 0.01$, the LOESS curve still closely follows the data pattern.

```
# plot
{
plot(xpts, ypts, xlab='x', ylab='y') # plot the original data points
lines(xpts, f, col="black", lty=1, lwd=2) # plot the true function
lines(y_loess$x0, y_loess$y0, col='red', lty=2, lwd=2) # plot the fitted model

title(main="fit with loess function")
legend("bottomright", legend = c("true f", "loess fit"), col = c("black", "red"), lty = c(1, 2))
}
```

fit with loess function



The above figure shows the original data points, the true regression function, and the fitted LOESS curve. The fitted curve achieves a good balance between bias and variance, effectively capturing the underlying pattern of the data. However, some deviation is observed near the peaks.

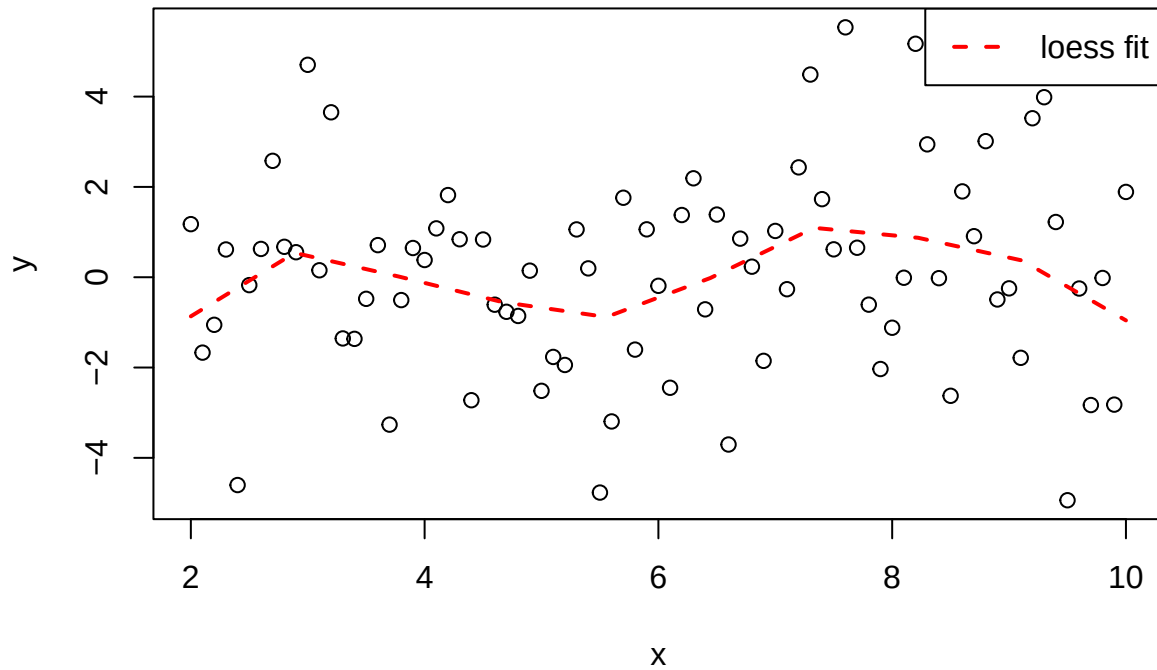
Case 3

In Case 3, I increase the variance to examine high-variance noise, using $\sin(x)$ as the true function.

```
xpts = seq(2, 10, 0.1) # x-values
f = sin(xpts) # true function
ypts = f + rnorm(length(xpts), 0, 2) # y-values
dat = data.frame(xpts, ypts)

y_loess = myloess(dat, 0.2, 10)
```

fit with loess function

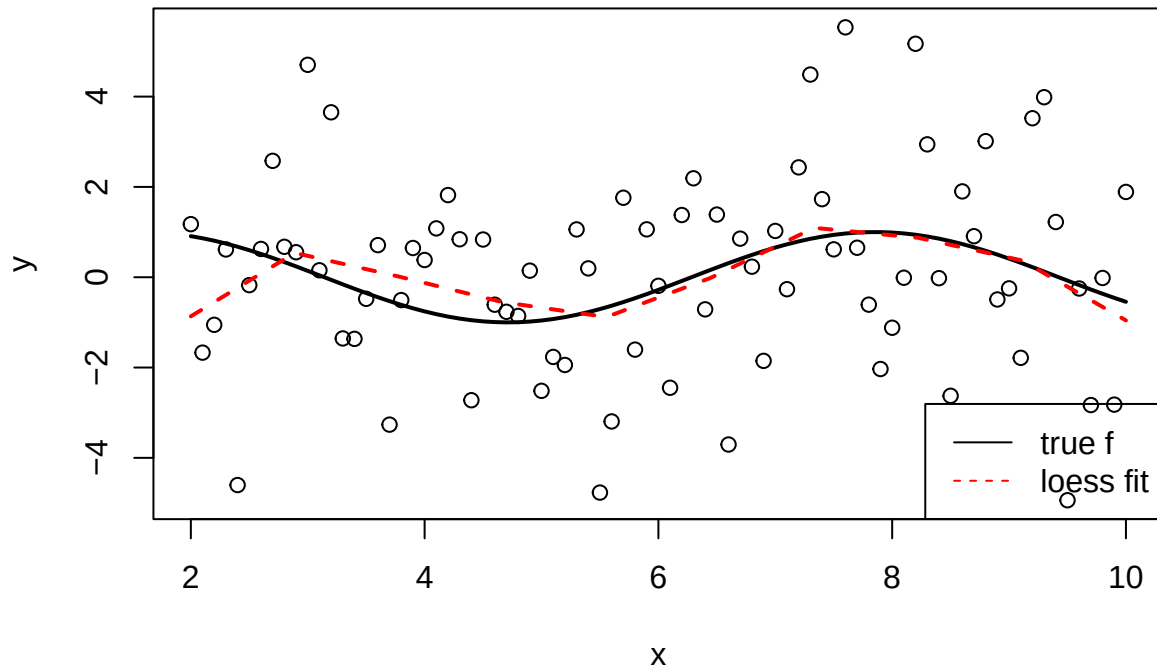


The figure presents the observed data points and the fitted LOESS curve under homogeneous high-variance noise.

```
# plot
{
plot(xpts, ypts, xlab='x', ylab='y') # plot the original data points
lines(xpts, f, col="black", lty=1, lwd=2) # plot the true function
lines(y_loess$x0, y_loess$y0, col='red', lty=2, lwd=2) # plot the fitted model

title(main="fit with loess function")
legend("bottomright", legend = c("true f", "loess fit"), col = c("black", "red"), lty = c(1, 2))
}
```

fit with loess function



The figure shows the original data points, the true regression function, and the fitted LOESS curve. Under high constant variance, the LOESS curve produces a smooth fit that captures the overall trend of the data. However, the fitted curve is still slightly biased, which could be further improved by increasing the number of data points.

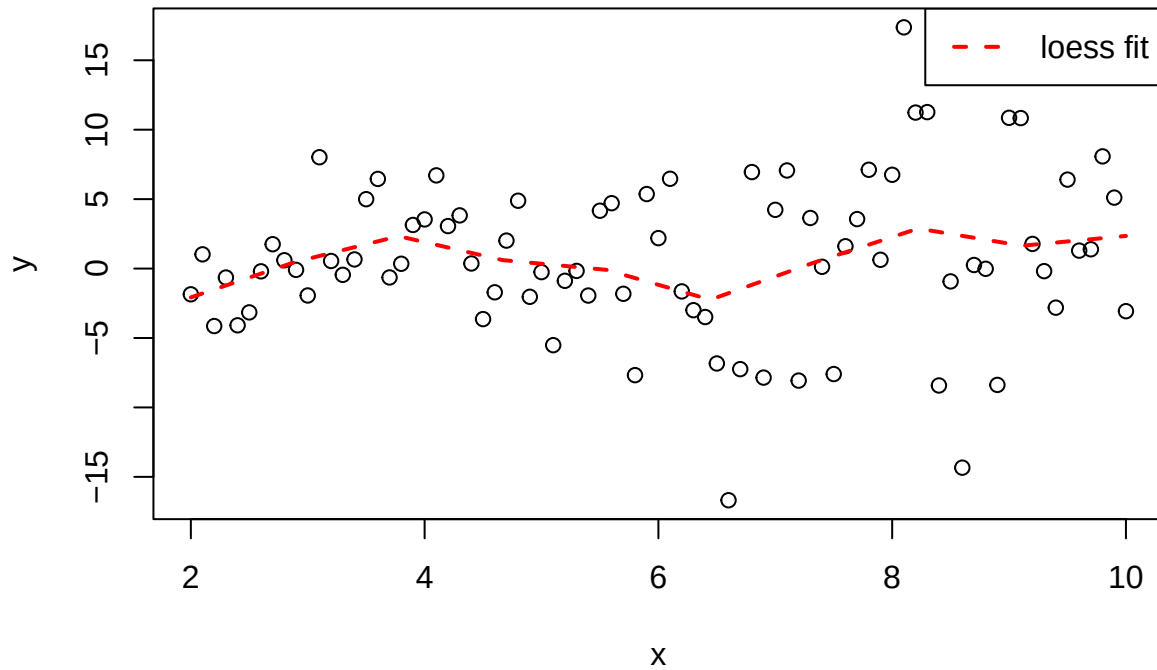
Case 4

In Case 4, I increase both the model complexity and the noise variance. In this case, I set heterogeneous variance and use $\sin(2x)$ as the true regression function.

```
xpts = seq(2, 10, 0.1) # x-values
f = sin(2*xpts) # true function
ypts = f + rnorm(length(xpts), 0, xpts) # y-values
dat = data.frame(xpts, ypts)

y_loess = myloess(dat, 0.2, 10)
```

fit with loess function

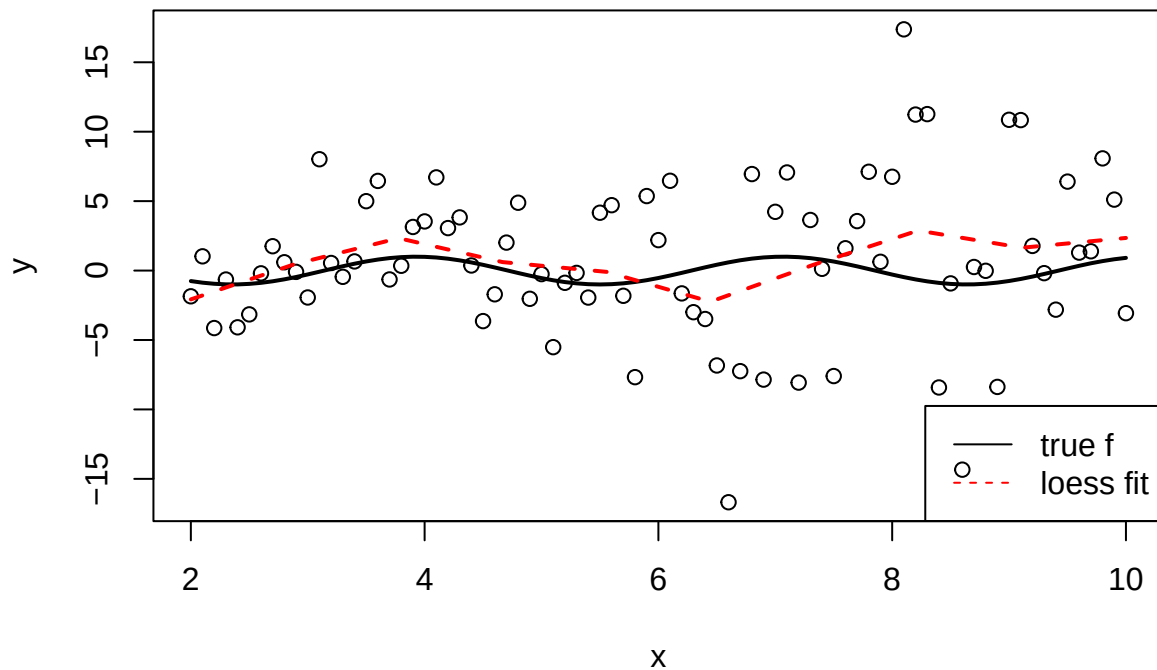


The figure shows the original data points and the fitted LOESS curve for heterogeneous variance.

```
# plot
{
plot(xpts, ypts, xlab='x', ylab='y') # plot the original data points
lines(xpts, f, col="black", lty=1, lwd=2) # plot the true function
lines(y_loess$x0, y_loess$y0, col='red', lty=2, lwd=2) # plot the fitted model

title(main="fit with loess function")
legend("bottomright", legend = c("true f", "loess fit"), col = c("black", "red"), lty = c(1, 2))
}
```

fit with loess function



The figure shows the original data points, the true regression function, and the fitted LOESS curve for data with heterogeneous variance. In the presence of heteroscedasticity, the LOESS curve captures the overall trend but becomes less reliable in regions with higher variability, where the fit is more influenced by noisy observations.

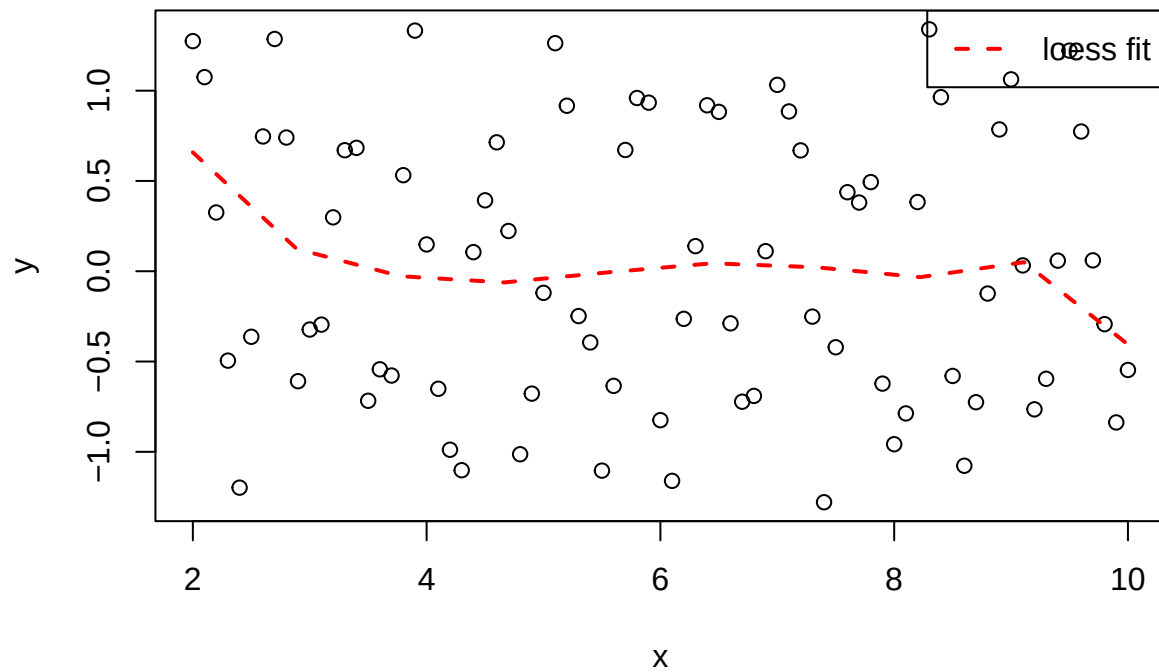
Case 5

In Case 5, the true function was $\sin(10x)$, introducing higher-frequency oscillations.

```
xpts = seq(2, 10, 0.1) # x-values
f = sin(10*xpts) # true function
ypts = f + rnorm(length(xpts), 0, 0.3) # y-values
dat = data.frame(xpts, ypts)

y_loess = myloess(dat, 0.2, 10)
```

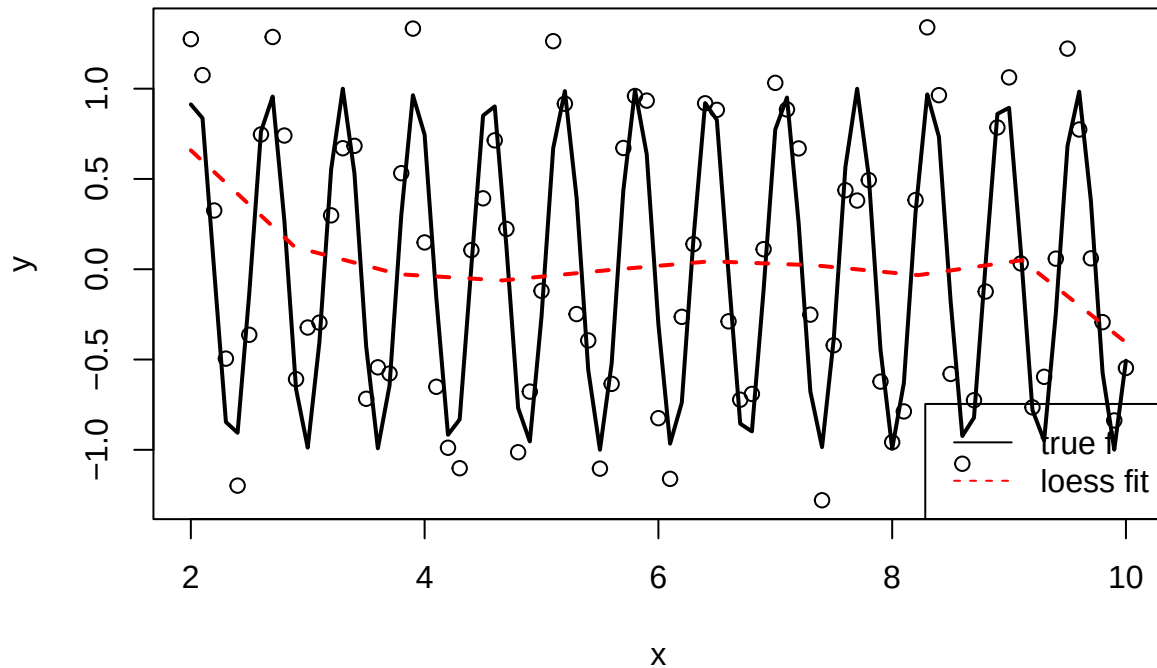
fit with loess function



```
# plot
{
plot(xpts, ypts, xlab='x', ylab='y') # plot the original data points
lines(xpts, f, col="black", lty=1, lwd=2) # plot the true function
lines(y_loess$x0, y_loess$y0, col='red', lty=2, lwd=2) # plot the fitted model

title(main="fit with loess function")
legend("bottomright", legend = c("true f", "loess fit"), col = c("black", "red"), lty = c(1, 2))
}
```

fit with loess function



In the presence of high-frequency, the LOESS method may fail to capture rapid oscillations in the true function, leading to an over-smoothed fit that does not reflect the local structure. This issue can be solved by selecting an optimal span value and a larger number of data points that presents the local region better.

In conclusion, the LOESS model performs locally weighted least squares regression, assigning high weight to nearby data points. As a result, it performs well when the true function is smooth and approximately linear within local regions. However, when the function oscillates rapidly within a local region, the LOESS method requires an appropriately chosen span to effectively capture the underlying pattern. On the other hand, LOESS tends to perform poorly when the data exhibit highly heterogeneous noise, as this can shift the fitted curve away from the true function toward the average of the noisy observations.

Q2

```
findspan <- function(dat, smoothvec, k, plotY) {  
  
  # The function takes the data, a vector of span values, and k as input,  
  # and returns the optimal span value based on k-fold cross-validation.  
  
  # data      : x and y values  
  # smoothvec : vector of span values  
  # k        : number of folds for k-fold cross-validation  
  # ploty    : binary indicator of whether to plot or not  
  
  colnames(dat) = c("x", "y")
```

```

dat = dat[order(dat$x), ] # sort data based on x-values

x = dat[, 1]
y = dat[, 2]

N = nrow(dat)
#folds = sample(rep(1:k, each=floor(N/k)+1)) # assign folds to data points
#folds = folds[1:N]
folds = sample(rep(1:k, length.out = N))
MSEvec = vector(length = length(smoothvec))

# loop over span values
for (s in 1:length(smoothvec)){
  MSE = 0
  # perform k-fold cross-validation
  for (f in 1:k){
    lfr = loess(y ~ x, data = dat[folds!=f, ], span = smoothvec[s]) # fit model
    pred = predict(lfr, newdata = data.frame(x = dat[folds==f, 1])) # perform prediction
    MSE = MSE + mean((dat[folds==f, 2] - pred)^2, na.rm = TRUE)
  }
  MSEvec[s] = MSE/k
}

# get the optimal span based on the minimum cross-validation MSE
best_span = smoothvec[which.min(MSEvec)]

if(plotY == TRUE){
  # plot the data with the optimal span
  plot(x, y, xlab='x', ylab='y') # plot the original data points
  model = loess(y~x, data = dat, span = best_span) # fit LOESS model with the optimal span using all
  lines(x, model$fitted, col='red', lty=2, lwd=2) # plot the fitted model
  legend("bottomright", legend = paste("optimal span:", best_span), col='red', lty = 2)
  #plot(smoothvec, MSEvec, type = 'c', xlab = "span", ylab = "MSE")
}

return(best_span)
}

```

Case 1

Here, I simulate $\sin(x)$ as the true function and add Gaussian noise $N(\mu=0, \text{sd}=0.3)$ to generate the response y . The `findspan` function is then applied, using `smoothvec` from 0.1 to 1 increase by 0.1, to fit the data via weighted least squares using k -fold cross validation.

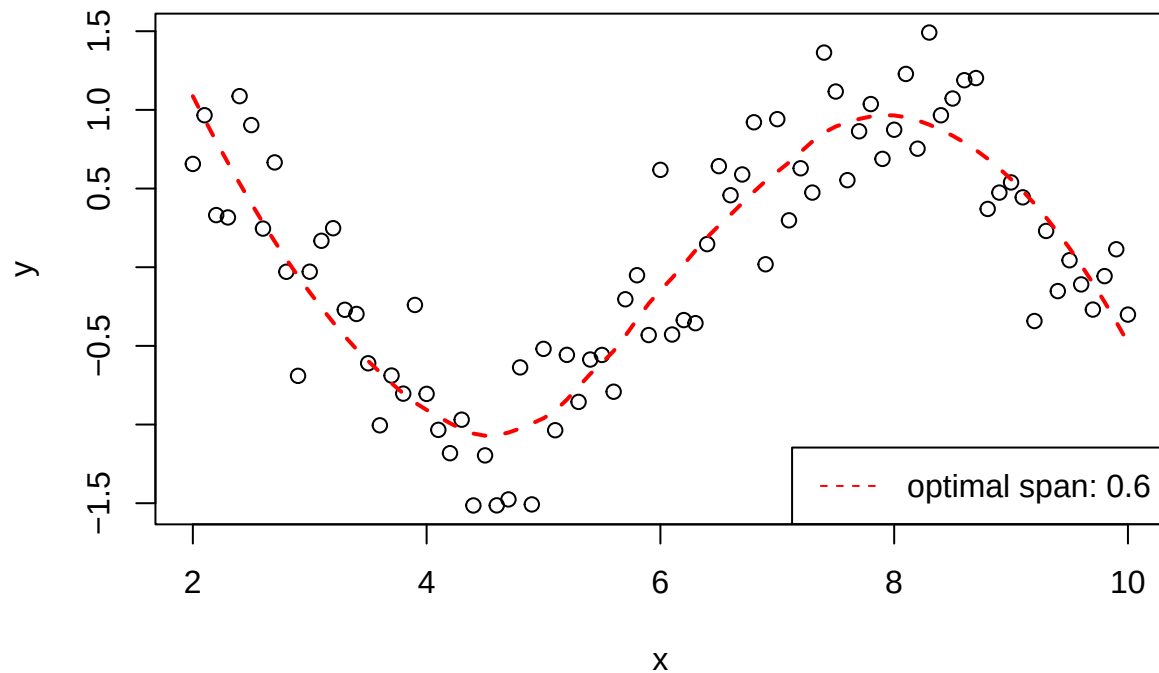
```

xpts = seq(2, 10, 0.1) # x-values
f = sin(xpts) # true function
ypts = f + rnorm(length(xpts), 0, 0.3) # y-values

dat = data.frame(xpts, ypts)
smoothvec = seq(from=0.1, to=1, by=0.1)
k = 5
plotY = TRUE

```

```
findspan(dat, smoothvec, k, plotY)
```



```
## [1] 0.6
```

With the optimal span value, which provides the lowest MSE in k-fold cross-validation, the LOESS method achieves a good balance between bias and variance, producing a smooth curve that closely follows the underlying trend of the data.

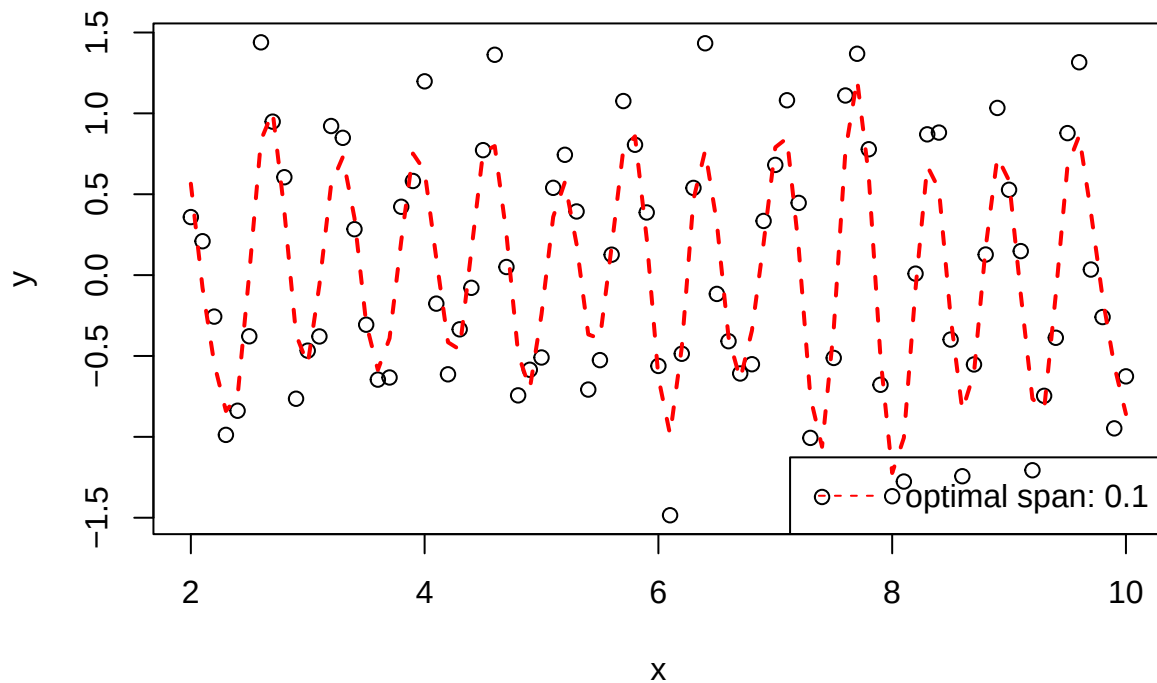
Case 2

In Case 2, the true function was made by using $\sin(10x)$, introducing higher-frequency oscillations.

```
xpts = seq(2, 10, 0.1) # x-values
f = sin(10*xpts) # true function
ypts = f + rnorm(length(xpts), 0, 0.3) # y-values

dat = data.frame(xpts, ypts)
smoothvec = seq(from=0.1, to=1, by=0.1)
k = 5
plotY = TRUE

findspan(dat, smoothvec, k, plotY)
```



```
## [1] 0.1
```

Here, we can see that the `findspan` function returns an optimal span value, which effectively captures the underlying trend of the data.

Q4

```
polls <- read.csv("HarrisTrumpPolls2024.csv", stringsAsFactors = FALSE, fileEncoding = "cp1252")
polls = polls[!is.na(polls$Sample), ]
final_data = data.frame()

for (i in 1:nrow(polls)) {

  pollster = polls[i, 1]
  date     = polls[i, 2]
  sample   = as.integer(strsplit(polls[i, 3], " ")[[1]][1])
  MOE      = polls[i, 4]
  Harris   = polls[i, 5]
  Trump    = polls[i, 6]
  tmp      = strsplit(date, "-")[[1]]
  start    = as.Date(paste0(tmp[1], "/2024"), "%m/%d/%Y")
  end      = as.Date(paste0(tmp[2], "/2024"), "%m/%d/%Y")
  days     = seq(start, end, by = "day")
```

```

w = sample / length(days)

# distribute over day
new_rows = data.frame(poll_date      = days,
                      pollster_name = pollster,
                      daily_sample   = w,
                      trump_support  = w * (Trump / 100),
                      harris_support = w * (Harris / 100))

final_data = rbind(final_data, new_rows)

}

```

Warning: NAs introduced by coercion

Warning: NAs introduced by coercion

```

#print(final_data)

result = aggregate(cbind(daily_sample, trump_support, harris_support) ~ poll_date,
                  data = final_data,
                  FUN = sum,
                  na.rm = TRUE)

result$day_of_year = as.numeric(format(as.Date(result$poll_date), "%j"))

total = result$daily_sample
result$harris_prop = result$harris_support / total
result$trump_prop  = result$trump_support / total
res = result[result$day_of_year >= 180, ]

# find optimal span value

k = 5
smoothvec = seq(from=0.1, to=1, by=0.1)
plotY = FALSE

# perform k-fold cross validation
span_harris = findspan(data.frame(res$day_of_year, res$harris_prop) , smoothvec, k, plotY)
span_trump  = findspan(data.frame(res$day_of_year, res$trump_prop) , smoothvec, k, plotY)

# loess model with optimal span value
loess_harris = loess(harris_prop ~ day_of_year , data =res, span = span_harris)
loess_trump  = loess(trump_prop ~ day_of_year , data =res, span = span_trump)

# plot
y_range <- range(c(res$harris_prop, res$trump_prop), na.rm = TRUE)

{
plot(res$day_of_year,
     res$harris_prop,
     pch = 1, col = "blue", ylim = y_range,

```

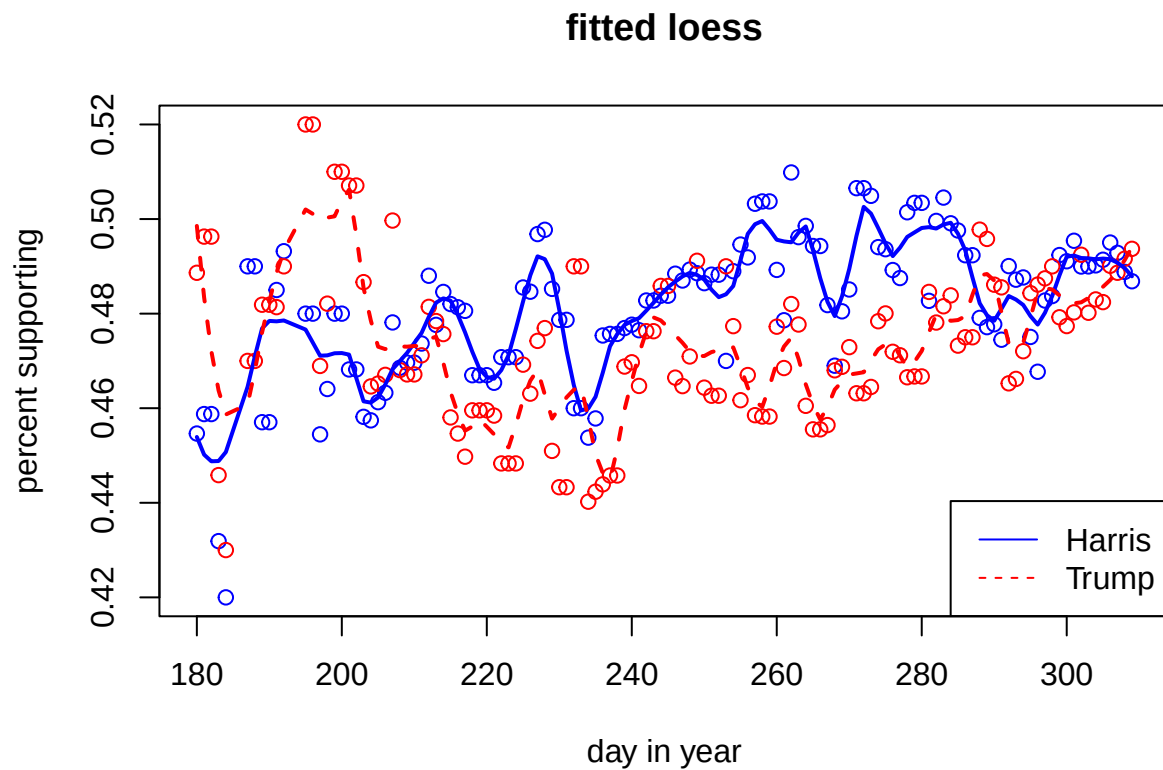
```

xlab = "day in year", ylab = "percent supporting")

points(res$day_of_year, res$trump_prop, pch = 1, col = "red")

lines(res$day_of_year, fitted(loess_harris), col="blue", lty=1, lwd=2)
lines(res$day_of_year, fitted(loess_trump), col="red", lty=2, lwd=2)
title("fitted loess")
legend("bottomright", legend = c("Harris", "Trump"), col=c("blue", "red"), lty=c(1,2))
}

```



```

cat("Best optimal span value for harris:", span_harris, "\n")

```

```

## Best optimal span value for harris: 0.1

```

```

cat("Best optimal span value for trump:", span_trump, "\n")

```

```

## Best optimal span value for trump: 0.1

```

```

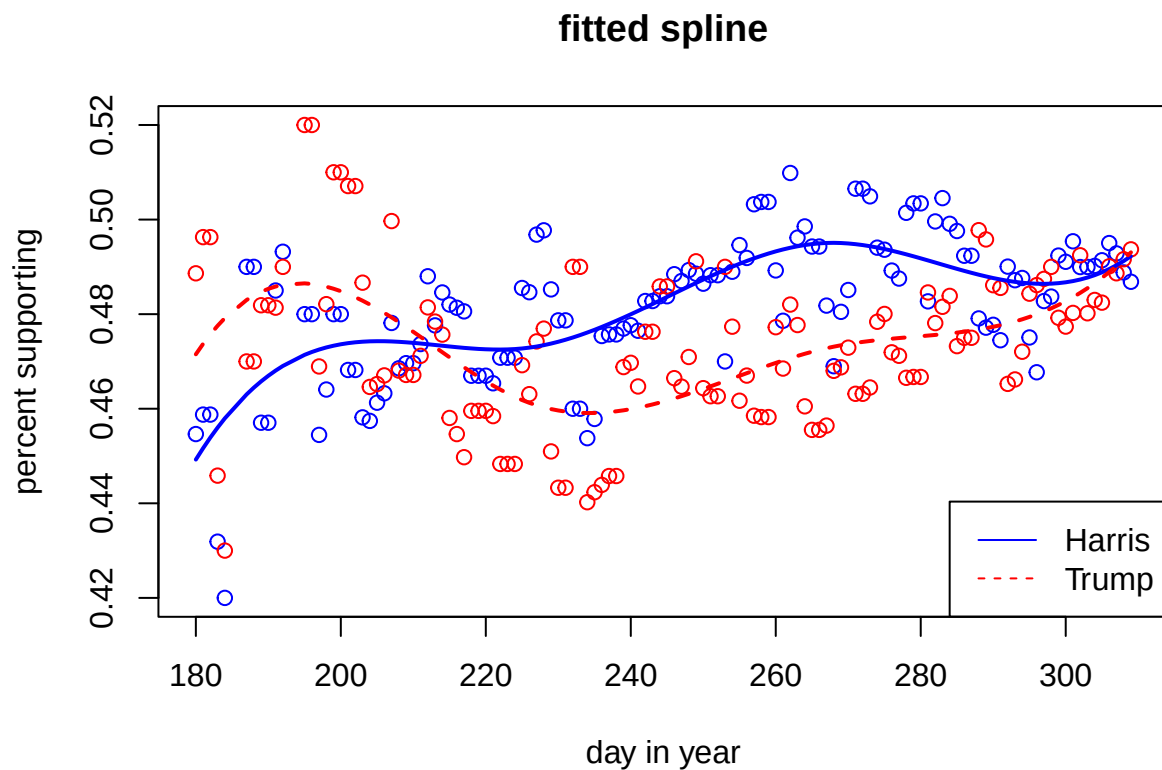
library(splines)
sp_harris = lm(harris_prop~bs(day_of_year,df=5) , data=res)
sp_trump = lm(trump_prop~bs(day_of_year,df=5) , data=res)

```

```
{
plot(res$day_of_year,
     res$harris_prop,
     pch = 1, col = "blue", ylim = y_range,
     xlab = "day in year", ylab = "percent supporting")

points(res$day_of_year, res$trump_prop, pch = 1, col = "red")

lines(res$day_of_year, fitted(sp_harris), col="blue", lty=1, lwd=2)
lines(res$day_of_year, fitted(sp_trump), col="red", lty=2, lwd=2)
title("fitted spline")
legend("bottomright", legend = c("Harris", "Trump"), col=c("blue", "red"), lty=c(1,2))
}
```



In this analysis, I fitted a LOESS model using cross-validated optimal span values and a spline with 5 degrees of freedom to estimate trends in daily polling averages. Both methods showed similar overall patterns, with LOESS capturing more local variation and the spline providing a smoother global trend. The plots indicated that Kamala Harris appeared to be ahead of Donald Trump for most of the observed days, although the gap became smaller toward the end of the year. However, Donald Trump won the election, suggesting that the polling averages overestimated Harris's support.